



WIF3009

Python for Scientific Computing

Data Manipulation and Visualization



Data Manipulation

Data Manipulation



- By now, you have heard quite a number of Python's data structures.
- Recall the regular Python List and NumPy array.
- Python List has some limitations such as it cannot perform element wise operations. This limitation is addressed by the NumPy array.
- NumPy array focuses on dealing with numerics. As such, NumPy only accepts a single data type in its array.

Data Manipulation



- In reality, data often comes in a tabular form consisting of columns with various data types.
- Pandas provide a perfect data structure for this situation using its data frame.
- It can hold different data types and works efficiently with a lot of data.
- It is built on NumPy and thus holds the same superiority on numerical data as the NumPy.

Data Manipulation (Pandas Data Frame)

- You can create Pandas data frame using its `DataFrame()` method.
- One of the ways to create a data frame is by passing a dictionary as the argument to `DataFrame()`.
- The keys represent the column names while values represent the data.

```
In [302]: import pandas as pd

In [303]: brics = pd.DataFrame({
    "country":["Brazil", "Russia", "India", "China", "South Africa"],
    "capital":["Brasilia", "Moscow", "New Delhi", "Beijing", "Pretoria"],
    "area":[8.516, 17.10, 3.286, 9.597, 1.221],
    "population":[200.4, 143.5, 1252, 1357, 52.98] })

In [304]: brics.head()
```

	area	capital	country	population
0	8.516	Brasilia	Brazil	200.40
1	17.100	Moscow	Russia	143.50
2	3.286	New Delhi	India	1252.00
3	9.597	Beijing	China	1357.00
4	1.221	Pretoria	South Africa	52.98

Data Manipulation (Pandas Data Frame)



- The data:

```
{  
    "country": ["Brazil", "Russia", "India", "China", "South Africa"],  
    "capital": ["Brasilia", "Moscow", "New Dehli", "Beijing", "Pretoria"],  
    "area": [8.516, 17.10, 3.286, 9.597, 1.221],  
    "population": [200.4, 143.5, 1252, 1357, 52.98]  
}
```

Data Manipulation (Pandas Data Frame)

- The integers on the left most column is the row index.
- Row index carry out similar function as column names, but for rows.
- You can rename row index using `.index` attribute.

```
In [305]: brics.index = ["BR", "RU", "IN", "CH", "SA"]
```

```
In [306]: brics.head()
```

```
Out[306]:
```

	area	capital	country	population
BR	8.516	Brasilia	Brazil	200.40
RU	17.100	Moscow	Russia	143.50
IN	3.286	New Delhi	India	1252.00
CH	9.597	Beijing	China	1357.00
SA	1.221	Pretoria	South Africa	52.98

Data Manipulation (Accessing Data Frame)

- You can access the data frame columns like you would with a regular Python List or NumPy array by using the square brackets [].
- For example, `brics['capital']`.
- It is worth noting that using a single square bracket like the example will return a Series.
- Series is just a 1D array with named index.
- To return a data frame, use double square brackets [[]].

```
In [431]: brics['capital']
```

```
Out[431]: BR      Brasilia  
          RU       Moscow  
          IN      New Delhi  
          CH       Beijing  
          SA       Pretoria  
          Name: capital, dtype: object
```

```
In [432]: brics[['capital']]
```

```
Out[432]:
```

	capital
BR	Brasilia
RU	Moscow
IN	New Delhi
CH	Beijing
SA	Pretoria

Data Manipulation (Accessing Data Frame)

- However, to access rows, it is not that straightforward.
- You cannot use the square bracket with the named index e.g. `brics['RU']`.
- You would have to use the integer indexes instead e.g. `brics[1:3]`.
- Also, this approach is only valid through slicing. `brics[1]` will yield an error.

```
In [313]: brics[0:3]
```

```
Out[313]:
```

	area	capital	country	population
BR	8.516	Brasilia	Brazil	200.4
RU	17.100	Moscow	Russia	143.5
IN	3.286	New Delhi	India	1252.0

Data Manipulation (Accessing Data Frame)



- As you can see, using the square brackets limit the functionality to access the data frame.
- Ideally, you would want to be able to access your data frame as you did with you NumPy array i.e. `numpy_array[rows, columns]`
- Fortunately, Pandas provide us with the `loc()` and `iloc()` method.
- `loc()` method accesses the data frame using label-based indexing.
- `iloc()` method accesses the data frame using integer position-based indexing.

Data Manipulation (Accessing Data Frame)

```
In [318]: brics.loc[["RU", "IN", "SA"], ["country", "population"]]
```

```
Out[318]:
```

	country	population
RU	Russia	143.50
IN	India	1252.00
SA	South Africa	52.98

RU	Russia	143.50
IN	India	1252.00
SA	South Africa	52.98

```
In [323]: brics.iloc[[0,2],[1,3]]
```

```
Out[323]:
```

	capital	population
BR	Brasilia	200.4
IN	New Delhi	1252.0

BR	Brasilia	200.4
IN	New Delhi	1252.0

```
In [319]: brics.loc[:, ["country", "population"]]
```

```
Out[319]:
```

	country	population
BR	Brazil	200.40
RU	Russia	143.50
IN	India	1252.00
CH	China	1357.00
SA	South Africa	52.98

BR	Brazil	200.40
RU	Russia	143.50
IN	India	1252.00
CH	China	1357.00
SA	South Africa	52.98

```
In [322]: brics.iloc[0:3,:]
```

```
Out[322]:
```

	area	capital	country	population
BR	8.516	Brasilia	Brazil	200.4
RU	17.100	Moscow	Russia	143.5
IN	3.286	New Delhi	India	1252.0

BR	8.516	Brasilia	Brazil	200.4
RU	17.100	Moscow	Russia	143.5
IN	3.286	New Delhi	India	1252.0

Data Manipulation (Grouping Data Frame)

- If you are familiar with databases, GROUP BY statement is a powerful feature to analyze your data.
- Pandas also offers this feature through its `groupby()` method.
- `groupby()` essentially splits data into different groups depending on a variable of your choice.
- For example, `phone_data.groupby('network')` groups the dataset by network.
- This method returns a GroupBy object which is a dictionary.

```
In [413]: phone_data.groupby('network').groups.keys()
```

```
Out[413]: dict_keys(['Meteor', 'Tesco', 'Three', 'Virgin', 'Vodafone'])
```

Data Manipulation (Grouping Data Frame)

- The most powerful feature of `groupby()` is that you can apply functions like `count()`, `max()`, `min()` to quickly calculate the values for each group.
- Once you have grouped the data, you specify the column that you want to calculate in square brackets and apply the appropriate function.

```
In [414]: phone_data.groupby('network')['duration'].sum()
```

```
Out[414]: network
          Meteor      18433.00
          Tesco         7.00
          Three         3.00
          Virgin      1775.00
          Vodafone    5164.35
          Name: duration, dtype: float64
```

Data Manipulation (Grouping Data Frame)

- You can also group your data based on a certain Boolean condition and apply calculations on them.
- For example, to find the sum of duration, for sms only, to each network:

```
In [426]: phone_data[phone_data['item'] == 'sms'].groupby('network')['duration'].sum()

Out[426]: network
          Tesco    289.0
          Three     3.0
          Name: duration, dtype: float64
```

Data Manipulation (Grouping Data Frame)

- To achieve a more complex query, you can group based on multiple columns.
- This will return a **multi index** dataframe.

```
In [427]: phone_data.groupby(['month', 'item'])['date'].count()

Out[427]: month      item      count
2014-11  call      107
          data       29
          sms       94
2014-12  call       79
          data       30
          sms       48
2015-01  call       88
          data       31
          sms       86
2015-02  call       67
          data       31
          sms       39
2015-03  call       47
          data       29
          sms       25
Name: date, dtype: int64
```

Data Manipulation (Grouping Data Frame)

- Recall that using a single square bracket will return a Series.
- You can use the **double square brackets** to get a data frame output when grouping your data.

```
In [433]: phone_data.groupby(['month', 'item'])[['date']].count()
```

```
Out[433]:
```

		date
month	item	
2014-11	call	107
	data	29
	sms	94
2014-12	call	79
	data	30
	sms	48
2015-01	call	88

Data Manipulation (Grouping Data Frame)

- You can get more complex results by applying multiple functions on your groupby output.
- This is done by using the `agg()` function.
- The following example calculates the maximum and sum of duration for each network provider.

```
In [434]: phone_data.groupby('network')['duration'].agg(['max','sum'])
```

```
Out[434]:
```

	max	sum
network		
Meteor	10528.000	18433.00
Tesco	2328.000	72402.00
Three	1.000	3.00
Virgin	174.000	1775.00
Vodafone	34.429	5164.35

Data Manipulation (Grouping Data Frame)

- Can even group your data and apply different functions to different columns!
- Use dictionary as the input.
- The example shows the sum of duration and the count of network type grouped by month and item.

```
In [436]: phone_data.groupby(['month', 'item']).agg({'duration': sum,  
                                                    'network_type': 'count'})
```

```
Out[436]:
```

		duration	network_type
2014-11	call	25547.000	107
	data	998.441	29
	sms	94.000	94
2014-12	call	13561.000	79
	data	1032.870	30
	sms	48.000	48
2015-01	call	17070.000	88
	data	1067.299	31
	sms	86.000	86

	month	item	duration	network_type
2014-11		call	25547.000	107
		data	998.441	29
		sms	94.000	94
2014-12		call	13561.000	79
		data	1032.870	30
		sms	48.000	48
2015-01		call	17070.000	88
		data	1067.299	31
		sms	86.000	86

Data Manipulation (Grouping Data Frame)

- The `agg()` function also accepts custom use defined function.
- The figure shows a custom function to find the data range for the duration.

```
In [437]: def data_range(series):  
          return series.max() - series.min()
```

```
In [439]: phone_data.groupby('network')['duration'].agg(data_range)
```

```
Out[439]: network  
Meteor      10525.0  
Tesco       2327.0  
Three        0.0  
Virgin      173.0  
Vodafone     0.0  
Name: duration, dtype: float64
```

Data Manipulation (Transpose)

- **Transposes** the index and columns of the DataFrame.
- It reflects the DataFrame over its main diagonal by writing rows as columns and vice-versa

```
import pandas as pd

df = pd.DataFrame({'Weight':[45, 88, 56, 15, 71],
                  'Name':['Sam', 'Andrea', 'Alex', 'Robin', 'Kia'],
                  'Age':[14, 25, 55, 8, 21]})

index_ = pd.date_range('2010-10-09 08:45', periods = 5, freq = 'h') |
df.index = index_
```

	Weight	Name	Age
2010-10-09 08:45:00	45	Sam	14
2010-10-09 09:45:00	88	Andrea	25
2010-10-09 10:45:00	56	Alex	55
2010-10-09 11:45:00	15	Robin	8
2010-10-09 12:45:00	71	Kia	21

Data Manipulation (Transpose)

- To transpose, just type: `dataframe.T`

df.T						
	2010-10-09 08:45:00	2010-10-09 09:45:00	2010-10-09 10:45:00	2010-10-09 11:45:00	2010-10-09 12:45:00	
Weight	45	88	56	15	71	
Name	Sam	Andrea	Alex	Robin	Kia	
Age	14	25	55	8	21	

Data Manipulation (Apply and Map)

- The `apply()` method is useful to apply a function along an axis of a data frame.
- In other words, you can quickly apply a function to the entire row or column. By default, the method applies the function to each column.

```
In [450]: df2
```

```
Out[450]:
```

	A	B
0	2	3
1	5	6
2	8	9

```
In [451]: df2.apply(np.mean)
```

```
Out[451]: A    5.0  
         B    6.0  
         dtype: float64
```

```
In [452]: df2.apply(np.mean, axis=1)
```

```
Out[452]: 0    2.5  
         1    5.5  
         2    8.5  
         dtype: float64
```

Data Manipulation (Apply and Map)

- If you want to apply a function only to specific columns, you can use the `map()` method.
- As a comparison, `map()` method works element-wise on a Series while `apply()` works on a row/column basis of a data frame.

```
In [453]: df2['B'].map(np.mean)

Out[453]: 0      3.0
          1      6.0
          2      9.0
          Name: B, dtype: float64
```

Data Manipulation (Combining dataframes)



- Pandas **dataframe.concat()** function is used to append rows of other data frames to the end of the given data frame, returning a new data frame object.
- Columns not in the original data frames are added as new columns and the new cells are populated with NaN value.

Data Manipulation (Combining dataframes)



```
Data1 = {"country": ["Brazil", "Russia", "India"],  
        "capital": ["Brasilia", "Moscow", "New Dehli"],  
        "area": [8.516, 17.10, 3.286],  
        "population": [200.4, 143.5, 1252] }
```

```
Data2 = {"country": ["Malaysia", "Thailand", "Vietnam"],  
        "capital": ["KL", "Bangkok", "Laos"],  
        "area": [6.516, 12.10, 4.286],  
        "population": [210.8, 172.1, 1471] }
```

Data Manipulation (Combining dataframes)

```
Data1 = {"country": ["Brazil", "Russia", "India"],
         "capital": ["Brasilia", "Moscow", "New Dehli"],
         "area": [8.516, 17.10, 3.286],
         "population": [200.4, 143.5, 1252] }

Data2 = {"country": ["Malaysia", "Thailand", "Vietnam"],
         "capital": ["KL", "Bangkok", "Laos"],
         "area": [6.516, 12.10, 4.286],
         "population": [210.8, 172.1, 1471] }

df1 = pd.DataFrame(Data1)
df2 = pd.DataFrame(Data2)
```

Data Manipulation (Combining dataframes)

- Without reindexing

```
pd.concat([df1, df2])
```

	country	capital	area	population
0	Brazil	Brasilia	8.516	200.4
1	Russia	Moscow	17.100	143.5
2	India	New Dehli	3.286	1252.0
0	Malaysia	KL	6.516	210.8
1	Thailand	Bangkok	12.100	172.1
2	Vietnam	Laos	4.286	1471.0

Data Manipulation (Combining dataframes)

- With reindexing

```
pd.concat([df1,df2], ignore_index=True)
```

	country	capital	area	population
0	Brazil	Brasilia	8.516	200.4
1	Russia	Moscow	17.100	143.5
2	India	New Dehli	3.286	1252.0
3	Malaysia	KL	6.516	210.8
4	Thailand	Bangkok	12.100	172.1
5	Vietnam	Laos	4.286	1471.0

Data Manipulation (Combining dataframes)

Outer join

`pd.concat([df1, df4], axis=1)`

df1					df4				Result							
										A	B	C	D	B	D	F
0	A0	B0	C0	D0	2	B2	D2	F2	0	A0	B0	C0	D0	NaN	NaN	NaN
1	A1	B1	C1	D1	3	B3	D3	F3	1	A1	B1	C1	D1	NaN	NaN	NaN
2	A2	B2	C2	D2	6	B6	D6	F6	2	A2	B2	C2	D2	B2	D2	F2
3	A3	B3	C3	D3	7	B7	D7	F7	3	A3	B3	C3	D3	B3	D3	F3
									6	NaN	NaN	NaN	NaN	B6	D6	F6
									7	NaN	NaN	NaN	NaN	B7	D7	F7

Data Manipulation (Combining dataframes)

Inner join

```
pd.concat([df1, df4], axis=1, join="inner")
```

df1					df4				Result							
	A	B	C	D		B	D	F		A	B	C	D	B	D	F
0	A0	B0	C0	D0	2	B2	D2	F2	2	A2	B2	C2	D2	B2	D2	F2
1	A1	B1	C1	D1	3	B3	D3	F3	3	A3	B3	C3	D3	B3	D3	F3
2	A2	B2	C2	D2	6	B6	D6	F6								
3	A3	B3	C3	D3	7	B7	D7	F7								

Data Manipulation (Combining dataframes)

Left join

```
pd.concat([df1, df4], axis=1).reindex(df1.index)
```

df1					df4				Result							
	A	B	C	D		B	D	F		A	B	C	D	B	D	F
0	A0	B0	C0	D0	2	B2	D2	F2	0	A0	B0	C0	D0	NaN	NaN	NaN
1	A1	B1	C1	D1	3	B3	D3	F3	1	A1	B1	C1	D1	NaN	NaN	NaN
2	A2	B2	C2	D2	6	B6	D6	F6	2	A2	B2	C2	D2	B2	D2	F2
3	A3	B3	C3	D3	7	B7	D7	F7	3	A3	B3	C3	D3	B3	D3	F3



Data Visualization

(with matplotlib)

Data Visualization (Principle of Analytics Graphics)



- Principle 1: Show comparisons
 - Evidence for a hypothesis is always *relative* to another competing hypothesis.
 - Always ask “Compared to What?”
- Principle 2: Show causality, mechanism, explanation, systemic structure.
 - What is your causal framework for thinking about a question?
- Principle 3: Show multivariate data.
 - Multivariate = More than 2 variables
 - The real world is multivariate
- Principle 4: Integration of evidence
 - Completely integrate words, numbers and diagrams.
 - Data graphics should make use of many codes of data presentation.

Data Visualization (Principle of Analytics Graphics)



- Principle 5: Describe and document the evidence with appropriate labels, scales, sources, etc.
 - A data graphics should tell a complete story that is credible.
- Principle 6: Content is king.
 - Analytical presentations ultimately stand or fall depending on the quality, relevance and integrity of their content.

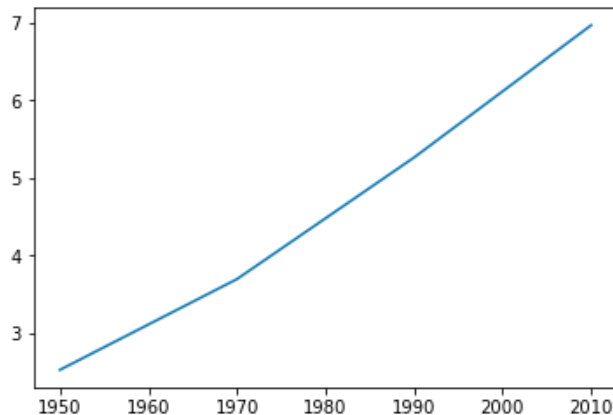
Data Visualization (Matplotlib)

- Data visualization is crucial in data analytics.
- It allows you to explore data, detect patterns and report interesting insights.
- A common visualization library in Python is the `matplotlib`.
- You need to import `matplotlib` before you can start using it in Python.

```
In [1]: import matplotlib.pyplot as plt

year = [1950, 1970, 1990, 2010]
pop = [2.519, 3.692, 5.263, 6.972]

plt.plot(year, pop)
plt.show()
```



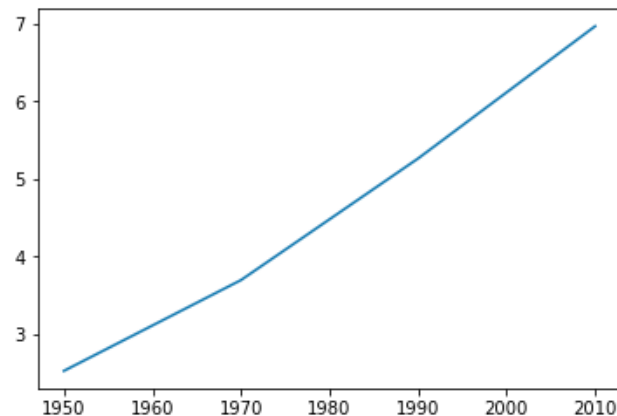
Data Visualization (Line plot)

- In matplotlib, the `plot()` method creates the graphic. However, it will not be displayed automatically to you (except you are in a Notebook environment)
- Therefore, you need to run the `show()` method to display the chart/graphic.

```
In [1]: import matplotlib.pyplot as plt

year = [1950, 1970, 1990, 2010]
pop = [2.519, 3.692, 5.263, 6.972]

plt.plot(year, pop)
plt.show()
```



Data Visualization (Scatterplot)

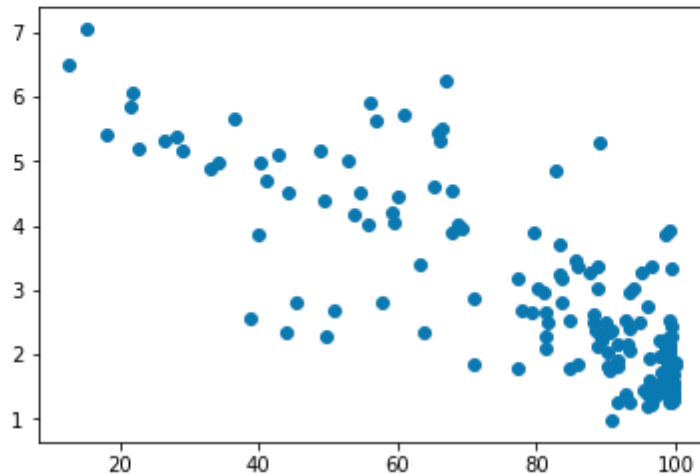


- A scatterplot is a useful summary of a **set of bivariate data** (two variables), usually drawn before working out a linear correlation coefficient or fitting a regression line.
- It gives a good visual picture of the **relationship between the two variables**, and aids the interpretation of the correlation coefficient or regression model.

Data Visualization (Scatterplot)

```
In [3]: import matplotlib.pyplot as plt
```

```
plt.scatter(population_literacy['Female Literacy'], population_literacy['Fertility'])  
plt.show()
```

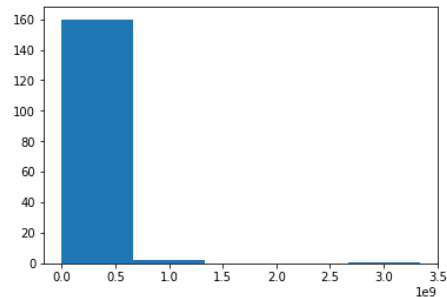


Data Visualization (Histogram)

- The horizontal axis is continuous like a number line
- The vertical axis is usually the frequency.
- You can set the number of bins (number of bars) via the `bins` argument.
- It is similar to a bar chart but a histogram groups numbers into ranges.

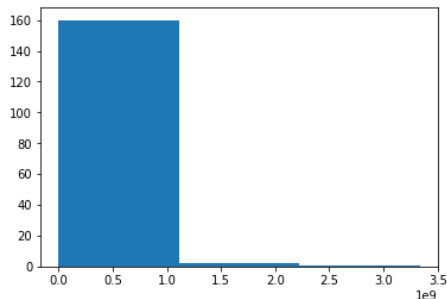
```
In [477]: plt.hist(data_pop['population'], bins=5)

Out[477]: (array([160.,  2.,  0.,  0.,  1.]),
          array([1.03566000e+05, 6.67309635e+08, 1.33451570e+09, 2.00172177e+09,
                2.66892784e+09, 3.33613391e+09]),
          <a list of 5 Patch objects>)
```



```
In [476]: plt.hist(data_pop['population'], bins=3)

Out[476]: (array([160.,  2.,  1.]),
          array([1.03566000e+05, 1.11211368e+09, 2.22412380e+09, 3.33613391e+09]),
          <a list of 3 Patch objects>)
```

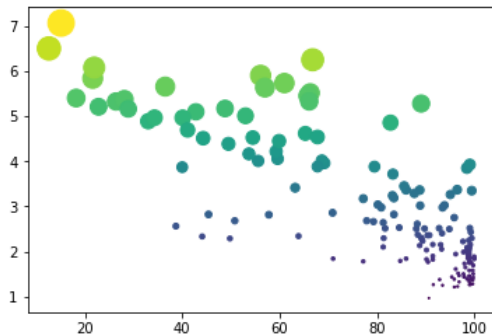


Data Visualization (Bubble Chart)

- Similar to scatter chart but the size of the markers differ based on their values.
- It is a more intuitive way of visualizing distribution.
- You can quickly detect the most important points by their sizes and colours.

```
In [486]: plt.scatter(data_pop['female literacy'], data_pop['fertility'], s=data_pop['fertility']**3, marker='o', c=data_pop['fertility'])
```

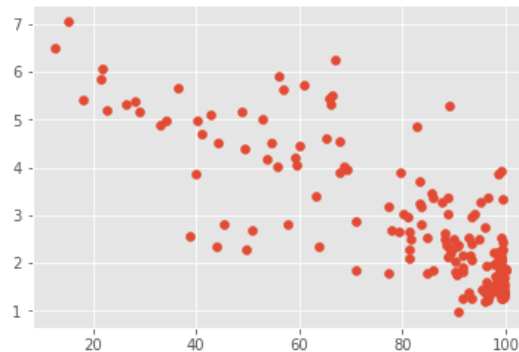
```
Out[486]: <matplotlib.collections.PathCollection at 0x122898c88>
```



Data Visualization (Emulating GGLOT)

- **ggplot** is a popular plotting package for R.
- It has nice visualization charts and is really popular among R users.
- Matplotlib allows user to emulate ggplot by setting the style sheet.
- You need to declare the ggplot style sheet before you start plotting you graphics.

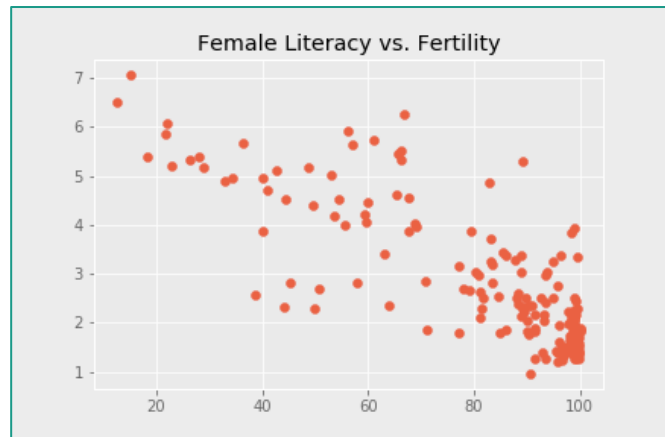
```
In [487]: plt.style.use('ggplot')  
  
In [488]: plt.scatter(data_pop['female literacy'], data_pop['fertility'])  
  
Out[488]: <matplotlib.collections.PathCollection at 0x1230ed518>
```



Data Visualization (Plot customization)

- Matplotlib provides a number of methods to customize your plots.
- This includes the option to add axis labels, renaming ticks and chart titles.
- The figure shows an example to add chart titles.
- Notice that you need to add the customization before you call the `show()` method.

```
In [500]: plt.scatter(data_pop['female literacy'], data_pop['fertility'])  
Out[500]: <matplotlib.collections.PathCollection at 0x1260f8e10>  
  
In [501]: plt.title('Female Literacy vs. Fertility')  
Out[501]: Text(0.5,1,'Female Literacy vs. Fertility')  
  
In [502]: plt.show()
```



Data Visualization (Plot customization)

- Other customization also applies the same principle.
- You need to do all the customization before you call the `show()` method.

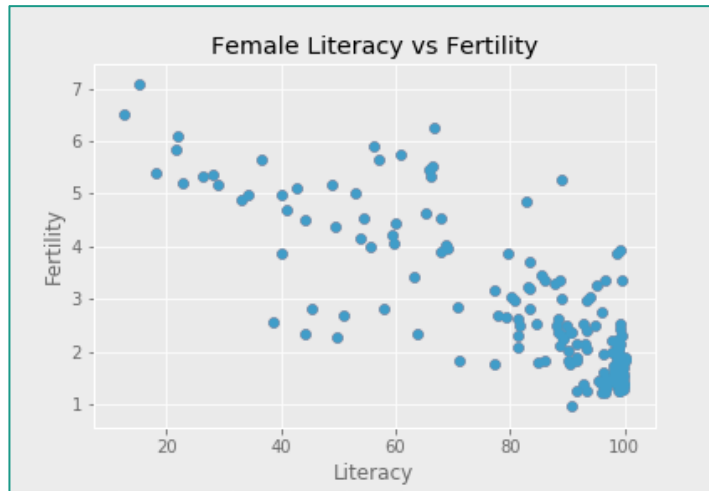
```
In [503]: plt.scatter(data_pop['female literacy'], data_pop['fertility'])
Out[503]: <matplotlib.collections.PathCollection at 0x128473668>

In [504]: plt.title('Female Literacy vs Fertility')
Out[504]: Text(0.5,1,'Female Literacy vs Fertility')

In [505]: plt.xlabel('Literacy')
Out[505]: Text(0.5,15,'Literacy')

In [506]: plt.ylabel('Fertility')
Out[506]: Text(36.625,0.5,'Fertility')

In [507]: plt.show()
```



Data Visualization (Multiple Subplots)



- When you call the `plot()` method e.g. `plt.plot()`, `plt.scatter()` etc., matplotlib implicitly creates a figure object behind the scenes.
- The figure object acts as a 'canvas' or 'drawing paper'. You then plot your charts on the 'canvas' or the so called figure.
- Figure can be explicitly created by calling `plt.figure()`.
- It is necessary to explicitly use `plt.figure()` when you want to tweak the size of the figure or adding multiple plots in a single figure.

Data Visualization (Multiple Subplots)



- On the other hand, subplots are multiple plots on a single figure.
- Imagine drawing multiple image on a single paper instead of using separate paper for each image.
- You can create subplots by using the `plt.subplots()` method.
- `subplots()` automatically create the figure object together with the number of subplots as defined through the arguments.
- You can access the subplots through the `subplot()` method (Note the singular !)

Data Visualization (Multiple Subplots)

- Notice how the subplots are accessed. `subplot(2,2,1)` accesses the first subplot in the 2x2 figure (upper left).
- `Subplot(2,2,2,)` refers to the upper right plot and so on.

```
In [543]: fig, ax = plt.subplots(nrows=2, ncols=2)
plt.subplot(2,2,1)
plt.scatter(data_pop['female literacy'], data_pop['fertility'])

plt.subplot(2,2,2)
plt.hist(data_pop['population'], bins=5)

plt.subplot(2,2,3)
plt.plot(range(10), range(10))

plt.subplot(2,2,4)
plt.scatter(data_pop['female literacy'], data_pop['fertility'], s=data_pop['fertility']**3, marker='o', c=data_pop['fertility'])

plt.show()
```



Data Visualization (Exporting plots)



- Exporting or saving plots to an external file is easy.
- Use the `savefig()` method and pass the output file name.
- The file type is defined by the extension that you input.
- For example, `plt.savefig('Figure1.png')` saves the plot as .png file.

Data Visualization (Visualizing using Plotly Express)



- Another common visualization library for Python is Plotly.
- Plotly allows pretty and interactive visualization.
- However, Plotly can sometimes be a bit intimidating due to the verbosity of the code.
- Therefore, we will explore a smaller module in Plotly known as Plotly Express.
- Plotly Express is a higher level wrapper for Plotly which allows **interactive visualization**.

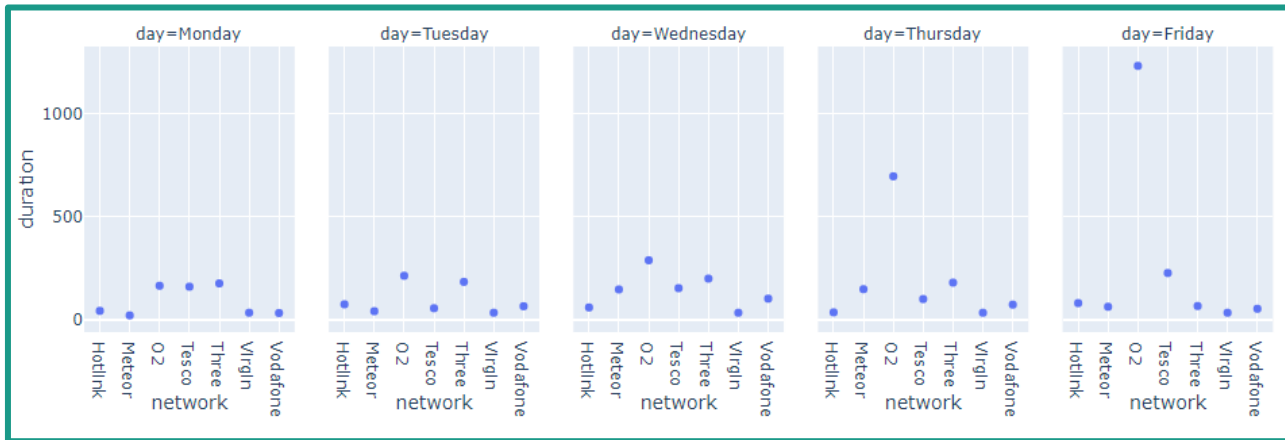
Data Visualization (Visualizing using Plotly Express)



- To use Plotly Express, you first need to install Plotly.
- You can use the `conda` package manager to install Plotly on your machine.
- `pip install plotly`
- To allow Plotly to work with Jupyter notebook, you need to **UNINSTALL** the Plotly extension (`jupyterlab-plotly`)
- Due to being deprecated already.

Data Visualization (Visualizing using Plotly Express)

- Plotly Express only needs a line of code to generate multiple subplots.
- Ccode below generates multiple plots to analyze the calling duration per day.
- `px.scatter(phone_data, x="network", y="duration", facet_col="day")`



Data Visualization (Visualizing using Plotly Express)



- For example, a standard line plot

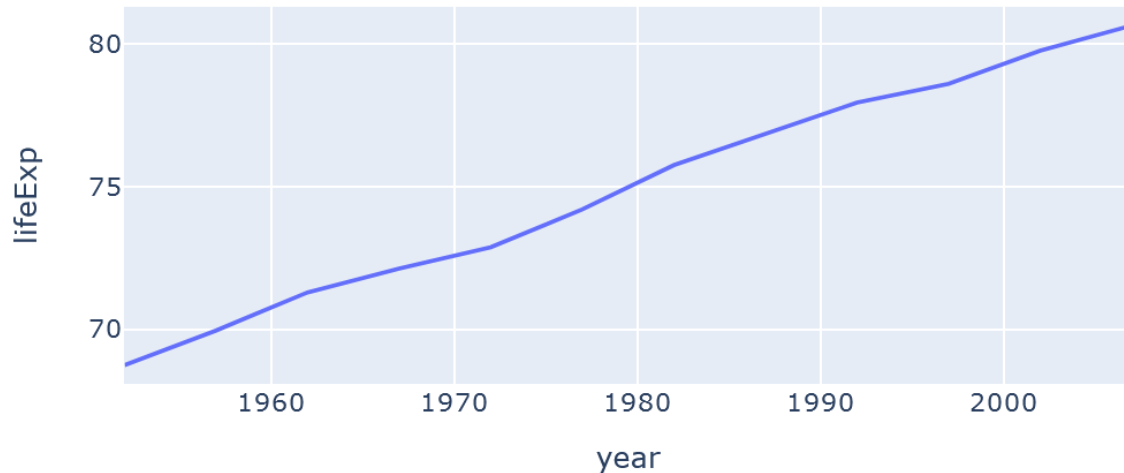
```
import plotly.express as px

df = px.data.gapminder().query("country=='Canada'")
fig = px.line(df, x="year", y="lifeExp", title='Life expectancy in Canada')
fig.show()
```

Data Visualization (Visualizing using Plotly Express)

- For example, a standard line plot

Life expectancy in Canada

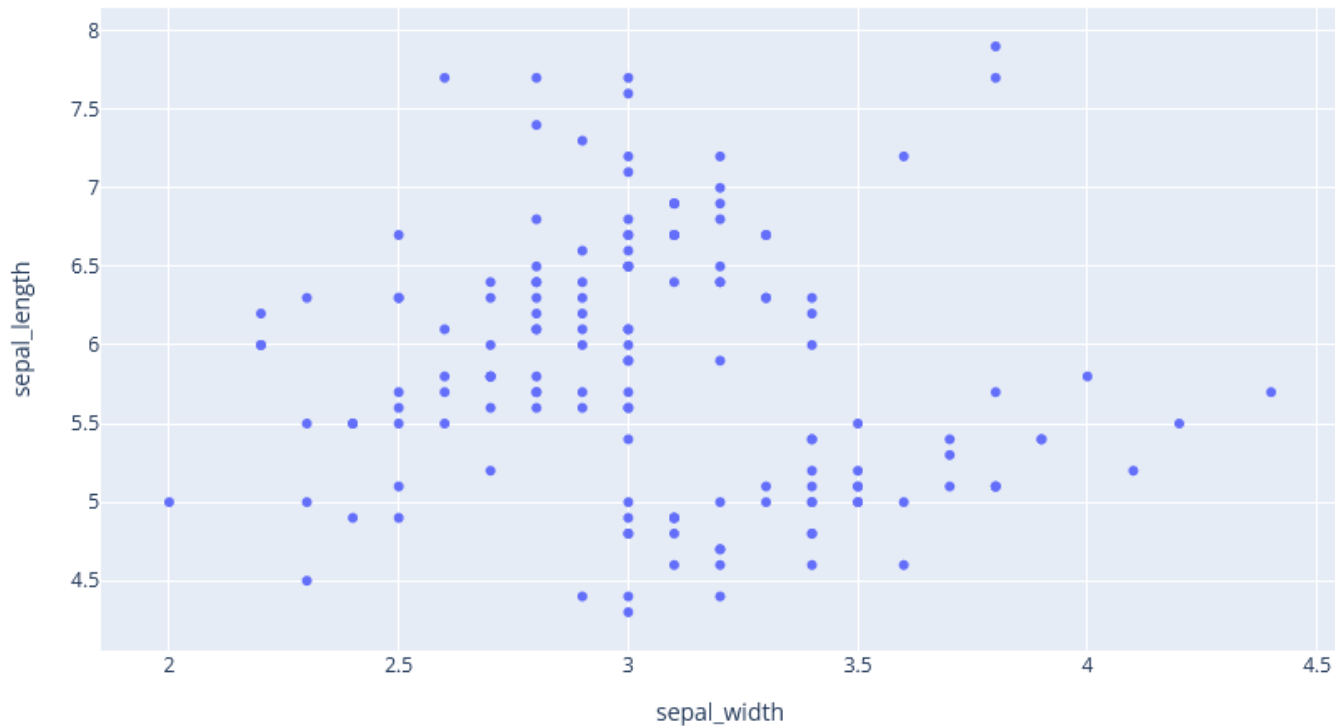


Data Visualization (Visualizing using Plotly Express)

- For example, a standard scatter plot

```
import plotly.express as px
df = px.data.iris() # iris is a pandas DataFrame
fig = px.scatter(df, x="sepal_width", y="sepal_length")
fig.show()
```

Data Visualization (Visualizing using Plotly Express)



Data Visualization (Visualizing using Plotly Express)

- You can also do 3D visualization as well!

```
import plotly.express as px
df = px.data.iris()
fig = px.scatter_3d(df, x='sepal_length', y='sepal_width', z='petal_width',
                    color='species')
fig.show()
```

Data Visualization (Visualizing using Plotly Express)

- You can also do 3D visualization as well!

